Team Name : **«TEAM NAME / ID»**

Team Leader Name : **Akash Boddula**

Problem Statement : **Intelligent Candidate Discovery & Ranking — rank the best-fit candidates for a job description by genuine fit, not keywords.**

Solution Overview

- **A hybrid candidate ranker.** It reads each candidate's full profile and ranks the top 100 by genuine fit — combining AI semantic understanding with a structured, recruiter-style scoring model.

- **Four layers working together:** (1) dense embeddings that grasp meaning, (2) structured fit rules, (3) integrity guards that reject fakes, (4) behavioural signals for real availability.

- **What makes it different:** AI-skill credit is GATED by title/career credibility — so a Marketing Manager who lists 'RAG' earns almost nothing, while a genuine engineer earns full weight.

- **Sees plain-language fits:** surfaces people who built ranking/retrieval systems but never wrote the buzzword — which keyword filters miss entirely.

- **Trustworthy & fast:** every pick has an explainable score + honest reason; runs in ~37s on a CPU with no paid LLM API calls.

JD Understanding & Candidate Evaluation

- **Requirements extracted from the JD:** production embeddings & vector search; a shipped ranking / recsys / search system at scale; product-company (not services-only); 5–9 yrs; evaluation rigor (NDCG/MRR/MAP); Noida/Pune; a pragmatic shipper, not research-only.

- **Most important candidate signals:** current + past job titles, what they actually built (career-history text), real retrieval/ranking evidence, experience band, product-vs-services history, and behavioural availability.

- **Fit beyond keywords:** we embed a distilled version of the JD and match each profile by meaning; we read career descriptions, not just the skills list; and we gate buzzword credit by whether the title/career makes it believable.

Ranking Methodology

- **Retrieve:** every profile is pre-embedded with a sentence-transformer (all-MiniLM-L6-v2, 384-d) and indexed with FAISS; we score cosine similarity to a distilled JD query.
- **Score (weighted blend 'fit'):** title .22 · what-they-built .22 · career-arc .14 · semantic-retrieval .13 · semantic-full .11 · experience .10 · location .08.
- **Combine all signals:** final = fit × behavioural × disqualifier × honeypot × stuffer — multipliers that sink unavailable, off-domain, fake, or impossible profiles.
- **Rank:** sort by final score (deterministic tie-break by candidate_id) and emit the top 100 with reasons.

Explainability & Data Validation

- **Every decision is explained:** each shortlisted candidate gets a 1–2 sentence reason built from their own facts — title, years, real employers, named skills — plus honest concerns (e.g. long notice, low response rate).

- **No hallucinations:** reasoning is composed only from fields actually present in the profile (no LLM, no invented skills); tone is tied to the rank so it never contradicts the score.

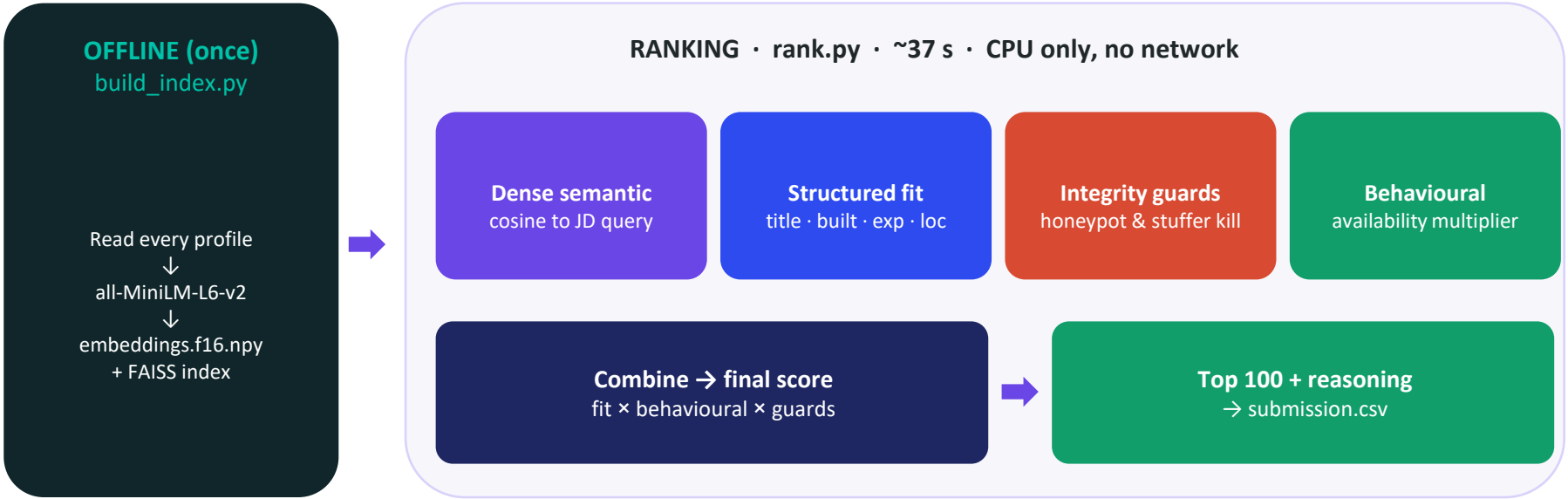
- **Suspicious / low-quality profiles:** honeypots (impossible tenure vs dates; 'expert' in many skills with 0 months used) are detected by simple arithmetic and crushed ×0.03; keyword-stuffers ×0.25–0.40.

- **Reproducible:** output is fully deterministic — two runs produce a byte-identical file.

End-to-End Workflow

- **From JD to ranked candidates:** one offline preparation step, then a fast CPU-only ranking pass.

System Architecture



Results & Performance

- **Ranking quality (released 100K pool):** top-100 is 100% engineering titles; 86/100 sit in the ideal 5–9-yr band (mean 6.5); Noida & Pune are the #1 and #2 locations — exactly the JD's preference.
- **Trap resistance:** 0 honeypots and 0 keyword-stuffers in the top 100 (the spec disqualifies any submission with >10% honeypots).
- **Meets every compute constraint:** ~37 s for 100K candidates on a 12-core CPU (limit is 5 min), ≤16 GB RAM, CPU-only, no network and no LLM API calls during ranking.
- **Production-shaped:** heavy embedding is a one-time offline step (~20 min, allowed); the ranking step is pure fast array math and is byte-for-byte reproducible.

Technologies Used

- **Python 3** — the whole system; clean and auditable.
- **sentence-transformers (all-MiniLM-L6-v2) + PyTorch (CPU)** — turns each profile into a meaning-vector so we match by intent, not keywords; small and CPU-friendly.
- **FAISS** — Facebook's similarity search; the vector-database / retrieval path.
- **NumPy** — all scoring is fast array math, which is why ranking takes seconds.
- **scikit-learn** — TF-IDF fallback so the pipeline runs even without the model present.
- **Streamlit** — the interactive sandbox demo. Why this stack: free, lightweight, CPU-only, no paid APIs, fully reproducible.

Submission Assets

- **GitHub repository:** «GITHUB REPO URL» (code, README, requirements, reproduce command).
- **Sandbox / demo:** «SANDBOX / DEMO URL (e.g. HuggingFace Space)» — or run locally with streamlit run app.py.
- **Ranked output:** submission.csv (top 100 candidates, validated format).
- **Reproduce command:** python rank.py --candidates ./candidates.jsonl --out ./submission.csv
- **Documentation:** approach deck + detailed project guide (PDF) included in /docs.

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU